

.NET VERSIONS, NAMESPACES, .NET CORE AND SOLUTIONS

1. Introduction

.NET is a software development platform created by Microsoft for building a wide range of applications, including web applications, Web APIs, desktop applications, cloud services, and other modern software systems. C# is one of the main programming languages used with .NET. Understanding the evolution of .NET, its versioning model, namespaces, .NET Core, and the concept of solutions provides an important foundation for learning C# and later developing applications with ASP.NET Core.

2. History and Versions of .NET

The .NET platform has evolved significantly since its original release. The early platform was known as .NET Framework and was closely associated with Windows. Microsoft later introduced .NET Core as a modern, open-source and cross-platform implementation. Starting with .NET 5, Microsoft unified the naming and moved forward with the modern .NET platform under the simple name '.NET'.

2.1 .NET Framework

.NET Framework was the original major implementation of .NET. It provided a runtime, class libraries, and development technologies for applications running primarily on Windows. Versions included .NET Framework 1.0, 1.1, 2.0, 3.0, 3.5, 4.x and later releases such as 4.8.1. Modern Microsoft guidance recommends using modern .NET for new development when possible, while .NET Framework remains important for maintaining existing Windows applications.

2.2 .NET Core

.NET Core was introduced as a redesigned generation of .NET with a strong focus on cross-platform development, performance, open-source development, and modern application scenarios. It could run on Windows, Linux, and macOS. Major releases included .NET Core 1.0, 1.1, 2.0, 2.1, 2.2, 3.0, and 3.1.

2.3 From .NET Core to Modern .NET

With .NET 5, Microsoft changed the product name from '.NET Core' to '.NET'. This represented the unification of the modern .NET platform and created a simpler versioning path: .NET 5, .NET 6, .NET 7, .NET 8, .NET 9, and .NET 10. Therefore, .NET Core should not be thought of as a completely separate technology from modern .NET; it is an important part of the platform's evolution.

Version	Release	Support Type	Status / End of Support
.NET Core 3.1	2019	LTS	Ended Dec. 13, 2022
.NET 5	2020	STS	Ended May 10, 2022
.NET 6	2021	LTS	Ended Nov. 12, 2024
.NET 7	2022	STS	Ended May 14, 2024
.NET 8	2023	LTS	Ends Nov. 10, 2026
.NET 9	2024	STS	Ends Nov. 10, 2026
.NET 10	2025	LTS	Ends Nov. 14, 2028

According to Microsoft, .NET releases follow a modern support lifecycle. Current supported releases include .NET 8 (LTS), .NET 9 (STS), and .NET 10 (LTS). Even-numbered releases are LTS releases, while odd-numbered releases are STS releases under the current release model.

3. .NET Versioning and Support

A .NET runtime version generally follows a major.minor.patch structure. The major version identifies the main release, while minor and patch versions provide additional features, fixes, and servicing updates. Microsoft releases a new major version of .NET every November.

Long Term Support (LTS) releases are intended to receive support for a longer period and are commonly preferred for applications that require a stable platform over several years. Standard Term Support (STS) releases have a shorter support period and provide access to newer features more frequently.

4. Namespace in C#

A namespace is a logical organizational structure used in C# to group related types such as classes, interfaces, enums, and other types. Namespaces help organize large applications and reduce naming conflicts between types that have the same simple name.

```
namespace MyApplication.Models
{
    public class Student
    {
        public string Name { get; set; }
    }
}
```

In this example, Student belongs to the MyApplication.Models namespace. Its fully qualified name can therefore be written as MyApplication.Models.Student.

4.1 Using Directives

The using directive allows a C# file to use types from a namespace without repeatedly writing the complete namespace path.

```
using MyApplication.Models;
```

```
Student student = new Student();
```

Without the using directive, the type could be referenced with its fully qualified name:

MyApplication.Models.Student. Namespaces and using directives therefore improve code organization and readability.

5. What is .NET Core?

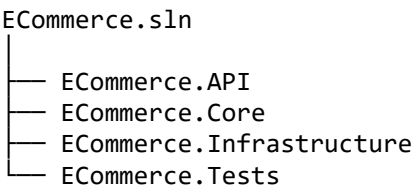
.NET Core was Microsoft's cross-platform implementation of .NET designed for modern software development. It was built with performance, modularity, open-source development, and cross-platform execution in mind. Developers could create applications that run on Windows, Linux, and macOS.

Important characteristics of .NET Core included cross-platform support, high performance, open-source development, a modular architecture, command-line tooling through the .NET CLI, and strong support for modern web development through ASP.NET Core.

After .NET Core 3.1, Microsoft released .NET 5 and dropped the 'Core' part of the name. Modern versions such as .NET 6, .NET 8, and .NET 10 are therefore the continuation of the modern .NET platform rather than separate products.

6. Solution in Visual Studio

A solution is a container used by Visual Studio to organize one or more related projects. A project represents a specific application or library and contains source code, configuration files, dependencies, and other resources required to build that project.



6.1 Solution vs. Project

Item	Purpose	Example
Solution	Organizes one or more related projects.	ECommerce.sln
Project	Contains the code and resources for a specific application or library.	ECommerce.API
Namespace	Organizes related C# types within code.	ECommerce.Models
Class	Defines an object/type and its behavior or data.	Customer

For example, a large .NET application may use separate projects for an API, business logic, data access, and automated tests. These projects can be placed inside one solution and connected through project references.

7. Relationship Between the Concepts

These concepts are connected but describe different levels of the .NET development environment:

- .NET is the overall modern development platform.
- .NET Core was the earlier name for the cross-platform modern generation that led to today's .NET.
- A Solution organizes one or more related projects in Visual Studio.
- A Project contains the source code, dependencies, configuration, and resources for an application or library.
- A Namespace organizes related C# types within a project or across an application.

7.1. SELF STUDY – Runtime Overhead and JIT

- **1. What is Runtime Overhead?**

- **Runtime overhead** refers to the additional time and system resources required by the runtime environment to perform tasks that are necessary for executing an application but are not part of the application's actual business logic.
- In .NET, one important source of runtime overhead is **JIT (Just-In-Time) Compilation**.

- **2. What is JIT Compilation?**

- When we write a C# program, the source code is first compiled into **Intermediate Language (IL)** rather than directly into machine code.
- The execution process can be represented as:
- **C# Code → IL → JIT Compiler → Machine Code → CPU**
- The **JIT Compiler** converts IL into native machine code that the CPU can execute. This compilation happens during application execution, which means it can introduce additional work at runtime.

- **3. Why Does JIT Cause Overhead?**

- JIT compilation requires CPU time and memory. When a method needs to be executed for the first time, the runtime may need to compile its IL into native machine code before executing it.
- As a result, JIT compilation can affect:
 - Application startup time
 - First-time execution performance
 - CPU usage
 - Memory usage
- After code has been compiled, the generated native code can generally be reused during that process, so the compilation cost does not need to be paid in the same way for every execution.

- **4. How Can This Overhead Be Reduced?**

- .NET provides different technologies and approaches to reduce the impact of JIT compilation.
- **1. ReadyToRun (R2R):**
ReadyToRun allows parts of an application to be precompiled into native code before execution. This can reduce the amount of work that JIT needs to perform at runtime and can improve application startup time.
- **2. Ahead-of-Time (AOT) Compilation:**
AOT compilation moves more of the compilation process from runtime to build time. Instead of relying heavily on JIT during execution, the application can be compiled into native code before it runs.
- **3. Native AOT:**
Native AOT compiles an application into native machine code during the build process. This can provide faster startup and reduce runtime compilation requirements, although it comes with some limitations and trade-offs compared with normal JIT-based .NET applications.

- **5. JIT vs AOT**

- | | |
|--|--------------------------------------|
| • JIT | • AOT |
| • Compilation happens during runtime | • Compilation happens before runtime |
| • Can introduce runtime compilation overhead | • Reduces runtime compilation work |

- **JIT**
 - Flexible and commonly used by .NET
 - Code can be optimized based on runtime information
 - **Conclusion**
 - JIT compilation is an important part of the traditional .NET execution model because it converts IL into machine code that the CPU can execute. However, performing compilation during runtime introduces some overhead. Technologies such as **ReadyToRun, AOT, and Native AOT** can reduce this overhead, particularly when faster startup time and reduced runtime compilation are important.
 - **Key Point:**
JIT = Compile during runtime
AOT = Compile before runtime
- **AOT**
 - Can provide faster startup
 - More work is performed during the build